

Lecture 38 (May 7)

Today we will briefly introduce chip-firing games and the Abelian sandpile model (ASM).

The ASM is a simple mathematical model, introduced by physicists (Bak, Tang, and Wiesenfeld in 1987), to study complicated natural processes like avalanches.

Later, Björner, Lovász and Shor studied the chip-firing game, which is a similar topic.

Chip-firing configurations

Suppose we have a connected graph $G = (V, E)$, with one vertex $q \in V$ designated the "sink". We will assume that the vertices are $V = \{0, 1, 2, \dots, n\}$ and the sink $q = 0$.

Definition (Configurations).

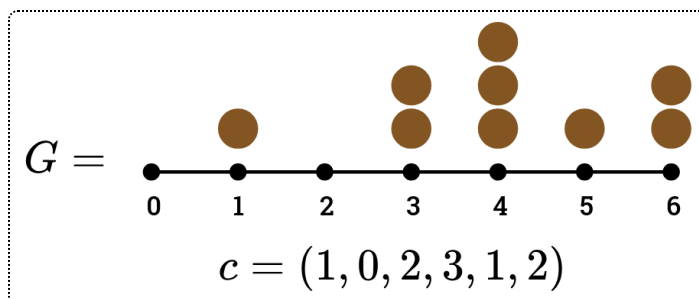
A **configuration** is a tuple of nonnegative integers

$$c = (c_1, c_2, \dots, c_n) \in \mathbb{Z}_{\geq 0}^n.$$

c_i can represent e.g. the number of chips or the number of snowflakes at location i .

In this class, we will refer to "chips" from the mathematical paper.

A configuration might look like this:



Definition (Stability).

A configuration c is **stable** if $c_i < d_i$ for all $i \in [n]$. In other words, every location (except the sink) has more neighbors than chips.

Otherwise, it is **unstable**.

For example, the above G is unstable at vertices 3, 4, and 6.

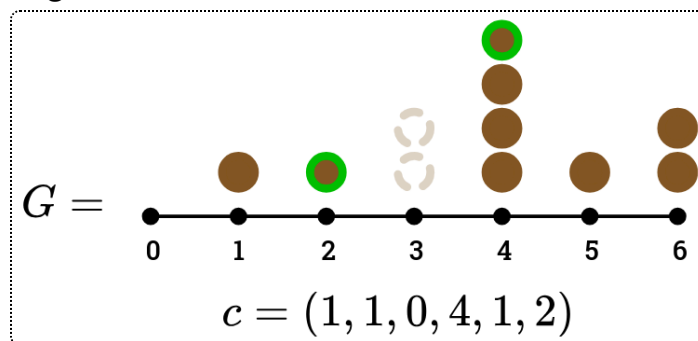
We have an operation on the configuration (firing chips or toppling snow) that can be done at unstable vertices.

Definition (Firing).

If $c_i \geq d_i$, we can **fire** vertex i by moving one chip to each neighbor of i .

Chips "disappear" if they are sent to the sink.

Here's what the configuration would look like if vertex 3 was fired.



We can do some more firings:

vertex fired	c after
(start)	$(1, 0, 2, 3, 1, 2)$
3	$(1, 1, 0, 4, 1, 2)$
4	$(1, 1, 1, 2, 2, 2)$
4	$(1, 1, 2, 0, 3, 2)$
3	$(1, 2, 0, 1, 3, 2)$
2	$(2, 0, 1, 1, 3, 2)$
1	$(0, 1, 1, 1, 3, 3)$
...	...

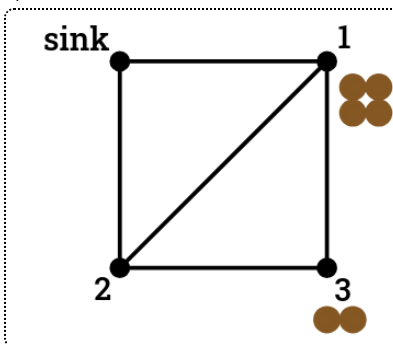
Note how in the last firing, one chip disappeared into the sink. There is a lemma that this process cannot go on forever (thanks to the sink).

Lemma 1 (Confluence).

For any initial configuration c_{init} :

- we always get a stable configuration c_{stab} after *finitely* many firings
- the result c_{stab} is unique (independent of the choice of firings).

As an example, for this initial configuration ($c_{\text{init}} = (4, 0, 2)$), no matter what, we will end up with $c_{\text{stab}} = (2, 2, 1)$.



Proof sketch of finiteness. Note that if every vertex is connected to the sink, then one chip disappears each time, so we easily get the result. This isn't true in general, but one chip always gets closer to the sink.

There is a way to come up with a nonnegative "value function" $f(c)$ on configurations, where each vertex is weighted in such a way that chips closer to the sink are worth less, so that every firing strictly decreases the function.

Proof of uniqueness. We will use induction on our value function $f(c)$ (assume that it exists and monotonicity is already proved). This idea is called a "diamond lemma" argument.

Suppose that c_{init} has two unstable vertices, i and j . We want to show that firing i and firing j will both yield the same unique stable configuration. (If there are more than two unstable vertices, just repeat this argument for all pairs.)

Let c' be the configuration after firing i , and c'' be that after firing j . By the inductive hypothesis, these two configurations will stabilize to some

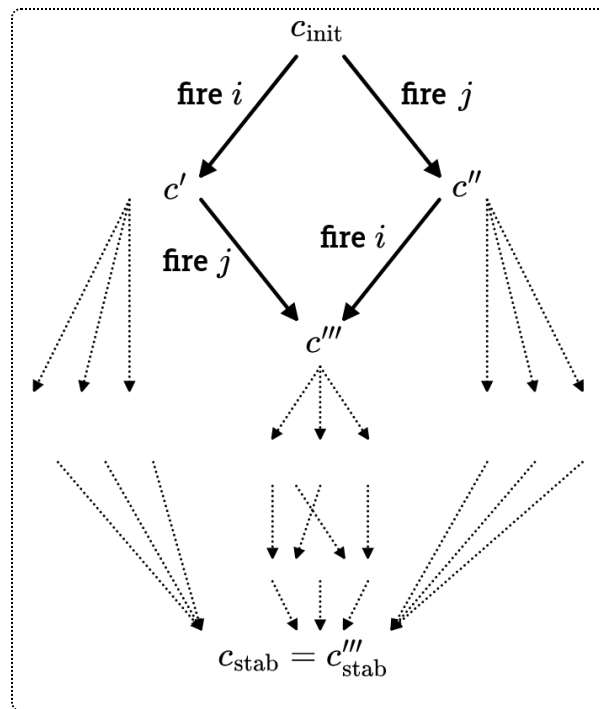
configurations c'_{stab} and c''_{stab} , respectively, no matter what. We want to prove that these are equal.

Note that j can still be fired in configuration c' : the number of chips c_j cannot have decreased after firing i . So we can fire j and get some configuration c''' .

Similarly, we can fire i in c'' . Note that this will yield the same configuration c''' .

Therefore, we must have $c'_{\text{stab}} = c'''_{\text{stab}} = c''_{\text{stab}}$. So, the process must result in the stable configuration c'''_{stab} regardless of whether we fire i or fire j first.

We can visualize this argument in a diagram (hence the name "diamond lemma"):

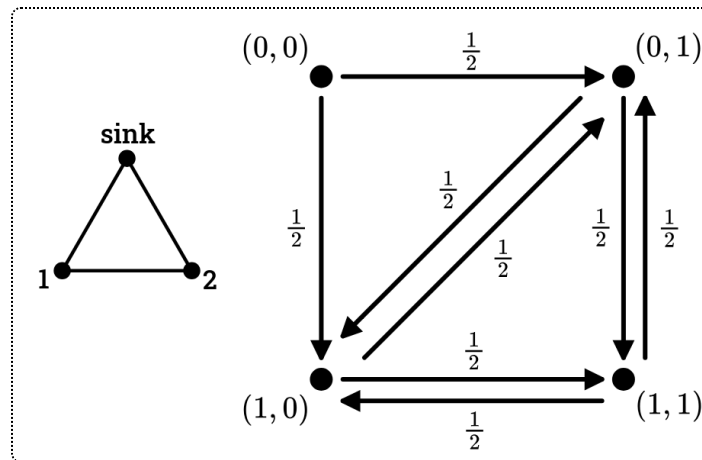


Abelian sandpile model

The Abelian sandpile model is a random walk/Markov chain on stable configurations c . Each step proceeds as follows:

- Randomly select a vertex $i \in \{1, 2, \dots, n\}$ (with uniform probability)
- Add a chip to c_i (and leave all other chips the same)
- If the result is not stable, stabilize it.

As an example, if $G = K_3$, there are four stable configurations, $(0, 0)$, $(1, 0)$, $(0, 1)$ and $(1, 1)$ (if any vertex has two chips, it can get fired). The state diagram is as follows:



With a Markov chain, we can compute the steady-state distribution: if we continue this process indefinitely, what does the probability of seeing each state end up being?

In the above, $(0, 0)$ has a probability of 0 (since it can't reoccur after the initial step) and all other configurations have a probability of $\frac{1}{3}$.

Definition (Recurrent configurations).

A stable configuration c is **recurrent** if, for any other stable configuration c' , there is a nonzero probability to go from c' to c .

$(0, 0)$ is not recurrent above since there's no way to go there from any other configuration. The other three configurations are recurrent.

Theorem 2 (About recurrent configurations).

- The steady-state distribution is the uniform distribution on the set of recurrent configurations.
- The number of recurrent configurations equals the number of spanning trees of the graph G .

An extension of the second claim:

Theorem 3 (Recurrent configurations and spanning functions).

For $G = K_{n+1}$, recurrent configurations $c = (c_1, \dots, c_n)$ are in bijection with parking functions $f = (f_1, \dots, f_n)$ (namely, with $f_i = n - c_i$).

For other graphs, there is a generalization of parking functions where an analogous bijection can be made.

Lecture 39 (May 9)

There are a lot of interesting things to learn about chip-firing games, but since we are nearing the end of the semester we will discuss some other topics.

(Sidenote: In the future, according to a department faculty meeting, math classes may be required to have some sort of exam. Do you think you'd prefer one final exam or three one-hour midterms? *[The class prefers midterms]*)

BEST Theorem

The BEST Theorem is named for de Bruijn, van Aardenne-Ehrenfest, Smith and Tutte, and was published in 1951.

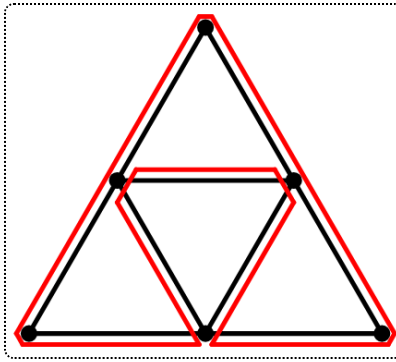
It relates to Eulerian tours:

Definition (Eulerian tour).

An **Eulerian tour** (or trail/path/circuit/cycle/walk) is a closed path in G (i.e. has the same start and endpoint) that traverses every edge exactly once.

(You may have heard of the origin story where Euler wanted to tour the seven bridges of Königsberg.)

For example, the red path is an Eulerian tour.



Theorem 4 (Euler (1736)).

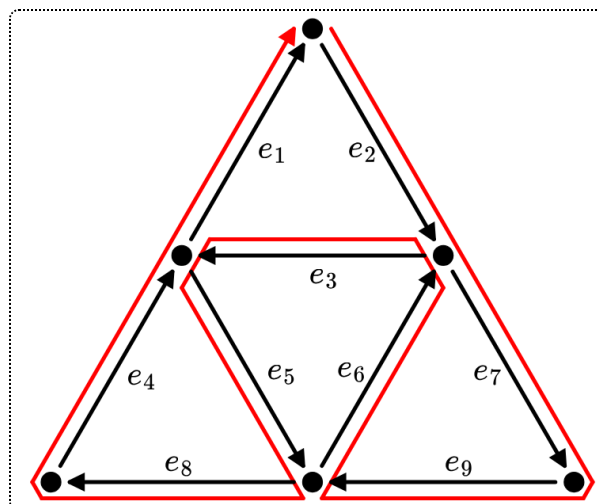
G has an Eulerian tour if and only if it is connected and every vertex has even degree.

It is not hard to see that having a tour implies those conditions, and the idea for constructing an Eulerian tour is by sequentially drawing arbitrary closed paths through unused edges and merging them together when they meet, until every edge is used exactly once.

We can also do the same on directed graphs (where our tour must follow edge directions), and the second condition becomes "every vertex has equal in-degree and out-degree". Directed graphs that have Eulerian tours are called **Eulerian digraphs**.

If we name all edges of an Eulerian digraph, note that an Eulerian tour is just an ordering of the set of edges E .

For example, with this example,



our cycle order is

$$C = e_2, e_7, e_9, e_6, e_3, e_5, e_8, e_4, e_1.$$

Note that there are $N = |E|$ cyclic shifts of C that we should consider equivalent, so let's fix an initial edge e_{init} and consider tours starting with this edge (not vertex!).

Recall that $\text{Arb}_r(G)$ is the number of arborescences in G with root r (spanning trees with all edges directed away from r). By the directed matrix-tree theorem, this is $\det(\tilde{L})$.

In an Eulerian digraph, $\text{Arb}_r(G) = \text{Arb}_{r'}(G)$ for any $r, r' \in V$.

Now we can formulate the BEST theorem:

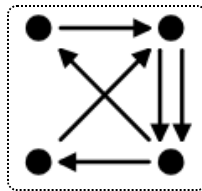
Theorem 5 (BEST theorem).

The number of Eulerian tours in an Eulerian digraph G , counted up to cyclic shifts, is

$$\text{Arb}_r(G) \cdot \prod_{v \in V} (\text{indeg}_G(v) - 1)!.$$

By convention, $0! = 1$.

For example, the following graph has two Eulerian tours.



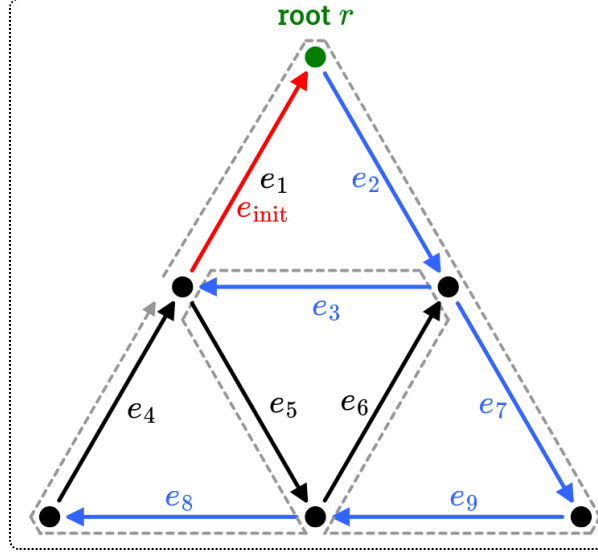
It has two arborescences rooted at each vertex and $2 \cdot 0! \cdot 1! \cdot 0! \cdot 1! = 2$.

Proof. We would like to find a bijection. First fix an initial edge e_{init} , and let r (the root) be its endpoint.

Then we claim that there is a bijection between Eulerian tours $C = (e_{i_1}, \dots, e_{i_N})$ with $e_{i_1} = e_{\text{init}}$, and tuples $(T, \{\pi_v\}_{v \in V})$, where T is an arborescence with root r

and each π_v is a permutation of size $\text{indeg}(v) - 1$. This immediately yields our expression.

For the forward direction, we mark the first edge in our Eulerian tour that enters each vertex $v \neq r$. For example, if our tour is $(e_1, e_2, e_7, e_9, e_6, e_3, e_5, e_8, e_4)$, we would mark these blue edges:



We claim that they form an arborescence rooted at r . Note that the number of edges is $|V| - 1$ and every non-root vertex has exactly one incoming edge (meaning that there are no directed cycles).

Then to obtain our permutations π_v , we simply take the non-marked edges coming into each vertex v in the order that they appear in C . Counting e_{init} as marked, there are indeed exactly $\text{indeg}_G(v) - 1$ unmarked edges ending at each vertex.

To reconstruct C from $(T, \{\pi_v\})$, we pick each edge in reverse order. We know where e_{i_N} must end (since it's the starting point of e_{init}); call it v . Then e_{i_N} must be the last edge in π_v . Similarly, we now know where $e_{i_{N-1}}$ ended (some vertex w , the origin of e_{i_N}), and we can take the last edge of w 's permutation. We repeat this, each time taking the last unused edge in the permutation, and if we used up all of the edges we take the one in the arborescence.

The reason we need the arborescence (instead of just having permutations of all $\text{indeg}_G(v)$ edges) is because the reverse process may get stuck in a cycle.

Lecture 40 (May 12)

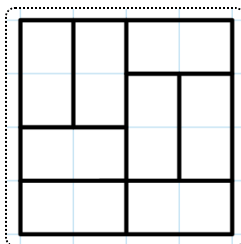
Today we will take a glimpse into another popular topic within combinatorics, namely tilings of various objects.

Domino Tilings

Definition (Domino tilings).

A **domino tiling** is a way to subdivide some (typically connected) region R into 2×1 and 1×2 dominoes without overlaps.

Here's an example domino tiling of a 4×4 square.



Some questions we can ask about regions:

- are they tileable?
- if so, how many ways are there to tile them?

Some aspects of statistical physics can be reduced to domino tiling problems, such as the Ising model for magnets, resolved by Kasteleyn and Temperley.

We'll look at some simple cases.

$2 \times n$ rectangles

For $n = 1, 2, 3, 4$, we can verify that there are 1, 2, 3, and 5 tilings of a $2 \times n$ rectangle, respectively.

Theorem 6 ($2 \times n$ tilings).

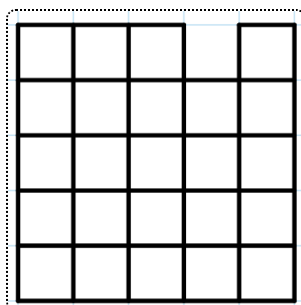
The number of tilings of a $2 \times n$ rectangle is the Fibonacci number F_{n+1} .

We need to prove that the recurrence $F_{n+1} = F_n + F_{n-1}$ applies to domino tilings. This is not hard to see: either the bottom-left corner needs to be covered by a vertical domino (which reduces to a $2 \times (n - 1)$ rectangle), or it needs to be covered by a horizontal domino (which forces another one above it and reduces to a $2 \times (n - 2)$ rectangle).

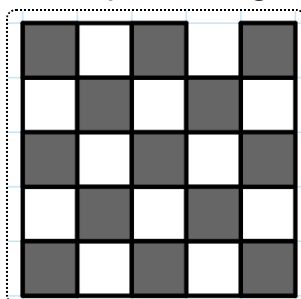


Rectangles

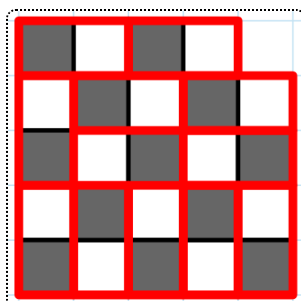
Let's consider a 5×5 square. Since there are an odd number of boxes, clearly there are no possible tilings. But what if we remove one of them, like this?



It turns out that even though there are 24 boxes, there is no way to tile this with twelve dominoes. This can be seen by shading in the boxes like a chessboard.

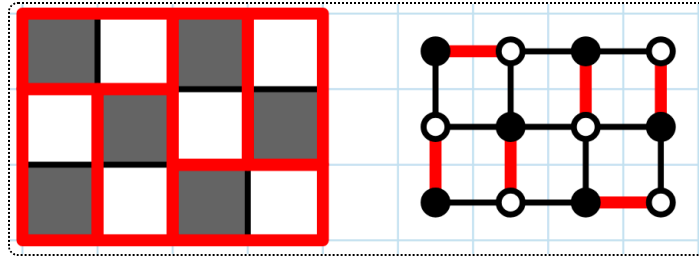


There are thirteen black boxes and eleven white ones, and we can verify that any domino must cover one of each color, so a tiling is impossible. However, if we remove a different box (like a corner) instead, it does become possible.



In general, this lets us reformulate our problem in terms of picking perfect matches within a bipartite graph. Namely, we let each box be a vertex and connect two vertices if they are adjacent. Then, no pair of vertices corresponding to boxes with the same color are adjacent.

Here is an example of this correspondence for a 3×4 rectangle.



Let the graph on the right be $G \subset K_{N,N}$, where N is the number of vertices of each color (6 here).

Consider the adjacency matrix of G . It looks something like this:

$$A = \begin{bmatrix} 0 & B \\ B^T & 0 \end{bmatrix}$$

where B is, in this case, (numbering black vertices 1, 2, 3, ... and white ones 1', 2', 3', ... in reading order):

	1'	2'	3'	4'	5'	6'
1	1	0	1	0	0	0
2	1	1	0	1	0	0
3	1	0	1	1	1	0
4	0	1	0	1	0	1
5	0	0	1	0	1	0
6	0	0	0	1	1	1

We can see that the number of perfect matchings or domino tilings is the "permanent" $\text{per}(B)$, which is like the determinant except we add all permutations (instead of subtracting half of them).

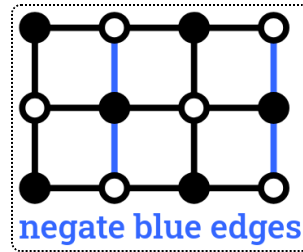
$$\text{per}(B) = \sum_{w \in S_N} \prod_{i=1}^n b_{i,w(i)}$$

To use this information, we need the following theorem.

Theorem 7 (Kasteleyn).

For a planar graph G , it is possible to modify B by reversing some signs of entries to obtain $\tilde{B}$, where $\text{per}(B) = \det(\tilde{B})$.

For a rectangular graph, we do this by negating the edges in alternating columns.



$$\tilde{B} = \begin{array}{c|cccccc} & 1' & 2' & 3' & 4' & 5' & 6' \\ \hline 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 2 & 1 & 1 & 0 & 1 & 0 & 0 \\ 3 & -1 & 0 & 1 & 1 & -1 & 0 \\ 4 & 0 & -1 & 0 & 1 & 0 & -1 \\ 5 & 0 & 0 & 1 & 0 & 1 & 0 \\ 6 & 0 & 0 & 0 & 1 & 1 & 1 \end{array}$$

With a lot of work, it is possible to determine the eigenvalues of this matrix and compute the determinant.

Theorem 8 (Kasteleyn (1961)).

Assume that n is even. Then the number of domino tilings of a $m \times n$ rectangle equals

$$\prod_{k=1}^{n/2} \prod_{l=1}^{\lfloor m/2 \rfloor} \left(4 \cos^2 \left(\frac{\pi k}{n+1} \right) + 4 \cos^2 \left(\frac{\pi l}{m+1} \right) \right)$$

Truncated rectangles

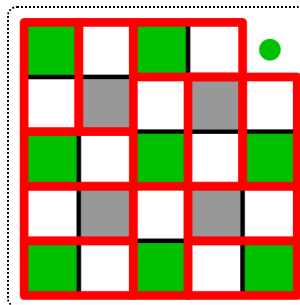
Now assume that both $m = 2k - 1$ and $n = 2\ell - 1$ are odd, and let's remove one box from a $m \times n$ rectangle. How many tilings are there now?

Theorem 9 (Temperley (1974)).

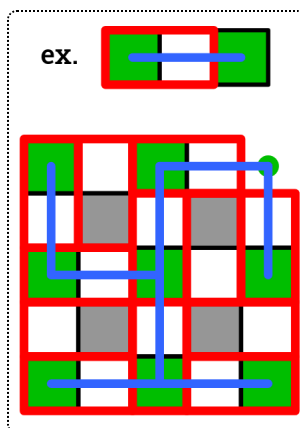
Let R be a $m \times n$ rectangle with box b removed, such that b belongs to the rectangle's boundary and has the same color as the corners (so that there are nonzero tilings).

Then the number of domino tilings of R equals the number of spanning trees of a $k \times \ell$ grid graph. (Note that this does not depend on the specific box removed.)

There is a nice bijection between domino tilings and spanning trees. For our example, let's use a 5×5 square missing a corner box. Here, we the shaded boxes in some alternating rows green — by definition, we removed a green box.



We can biject this to spanning trees on the grid graph of the green boxes. We add an edge (u, v) to the spanning tree if the domino containing u "points at" v , or vice versa, like so:



We can uniquely construct a tiling from a spanning tree by starting from the missing box and expanding outwards.

This also shows us how to explicitly convert between tilings where different green boxes are removed. We can form a directed path between any two green boxes, and we simply slide the dominoes along that path. For example, if had removed the top-left corner instead, we could do this:

